

Laboratorio 4 – Definición de clases

En esta sesión construiréis un programa formado por más de un archivo java. En uno de ellos definiréis una clase sencilla (Complex) para gestionar números complejos. En otra incorporareis el método main() para generar un programa interactivo que permita calcular operaciones sencillas con números complejos.

Objetivos

Los objetivos específicos de esta sesión son:

- Crear una clase sencilla desde cero.
- Analizar cómo se crean los objetos con el operador new y cómo los constructores inicializan sus atributos.
- Generar métodos de cálculo con números complejos.
- Generar el método toString() para construir un String que contenga representación textual de los valores de los atributos de los objetos (en el caso de Complex, que genere strings del tipo "3+4j" o "2-5j").
- Generar un pequeño menú de opciones para poder invocar operaciones desde la consola-

Inicio

La Figura 1 muestra el diagrama UML de la clase Complex que usaremos para gestionar y operar con números complejos. Para más información sobre dichos números y sobre UML podéis consultar las referencias que siguen:

http://ca.wikipedia.org/wiki/Nombre_complex

http://ca.wikipedia.org/wiki/Llenguatge_unificat_de_modelat

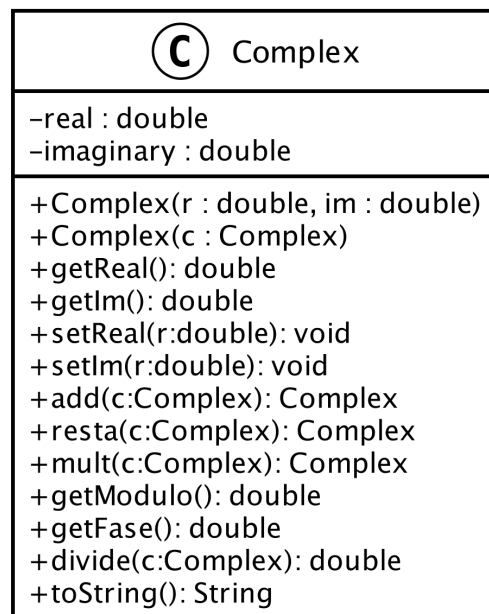


Figura 1. Diagrama UML de la clase Complex

A continuación se muestra una breve descripción de los métodos:

`public Complex(double r, double im): constructor;` inicializa la parte real al valor del argumento `r` y la parte imaginaria al valor de `im`.

`public Complex(Complex c): constructor;` inicializa los valores de la parte real e imaginaria a los valores de la parte real e imaginaria del argumento `c`.

`public double getReal():` operación de acceso para inspección; devuelve el valor de la parte real.

`public double getIm():` operación de acceso para inspección; devuelve el valor de la parte imaginaria.

`public void setReal(double r):` operación de acceso para asignación; asigna a la parte real el valor del

argumento `r`.

`public void setIm(double im):` operación de acceso para inspección; asigna a la parte real el valor del argumento `im`.

`public Complex add(Complex c):` Crea y devuelve un nuevo objeto `Complex` representando al número complejo que resulta de sumar el número complejo representado por el objeto que contiene el método y el argumento `c`.

`public Complex resta(Complex c):` Crea y devuelve un nuevo objeto `Complex` representando al número complejo que resulta de restar el número complejo representado por el objeto que contiene el método y el argumento `c`.

`public Complex mult(Complex c):` Crea y devuelve un nuevo objeto `Complex` representando al número complejo que resulta de multiplicar el número complejo representado por el objeto que contiene el método y el argumento `c`.

`public Complex divide(Complex c):` Crea y devuelve un nuevo objeto `Complex` representando al número complejo que resulta de dividir el número complejo representado por el objeto que contiene el método y el argumento `c`.

`public String toString():` Genera un nuevo `String` que muestra la representación textual de los atributos según la convención aritmética ampliamente aceptada: “3+4j”, “3-4j”, “-2-5j”. Se hace notar que al solicitar a Java la representación textual de un entero negativo, Java genera el signo -, pero cuando se le solicita la de un número positivo, Java NO muestra el signo +.

`public double getModulo():` operación de cálculo de una propiedad; devuelve el valor del módulo del número complejo. La clase `Math` contiene la implementación de las funciones que necesitáis para calcularlo (la raíz cuadrada por ejemplo).

`public double getFase():` operación de cálculo de una propiedad; devuelve el valor de la fase del número complejo. La clase `Math` contiene la implementación de las funciones trigonométricas.

Convenciones en código Java

Los programadores de Java siguen una serie de convenciones que ayuda a generar programas más legibles. Podéis encontrarlas en:

<https://www.oracle.com/technetwork/java/codeconventions-150003.pdf>

De entre ellas cabe destacar que los nombres de TODAS las clases de Java utilizan la convención conocida como `UpperCamelCase`, consistente en que la primera letra de todas las palabras que conforman el nombre es siempre MAYÚSCULA.

Por otro lado, los nombres de los atributos, métodos, argumentos y variables locales se eligen siguiendo la convención `lowerCamelCase`. En ella, cuando un nombre es la concatenación de varias palabras, la letra inicial de la primera es SIEMPRE minúscula, y las letras iniciales de las restantes palabras se escriben SIEMPRE CON MAYÚSCULAS. Esas mayúsculas ayudan a identificar el comienzo de cada palabra en el nombre. Se evita así el carácter subrayado `_` ampliamente utilizado en C.

Tareas a realizar

Tarea 1

Cread un nuevo proyecto. Cread en él el package “`edu.upc.etsetb.poo.math`”. Como sabéis, este package se crea en el directorio `src`.

Cread la clase `Complex` en dicho package.

Cread ahora el nuevo subdirectorio “`test`” en el directorio del proyecto. Podéis hacerlo de dos formas:

1. Navegad por el árbol de directorios hasta el directorio del proyecto con el navegador del sistema de archivos y una vez allí cread el directorio `test`, O BIEN
2. En NetBeans, situaos sobre “`Source packages`” en el navegador de proyectos. Pulsad botón derecho del ratón y seleccionad `New->Other->Unit Tests->JUnit Test`. Se os abrirá un cuadro de diálogo para crear una clase de pruebas. Seleccionad `CANCEL` para no crearla.

Una vez finalizado el proceso, en el navegador de proyectos observaréis que se os ha creado un nuevo

elemento: "Test Packages".

Cread allí el package "edu.upc.etsteb.poo.math.test".

Cread ahora la clase Test en dicho package. Es allí donde depositaréis el código de prueba, que de esta forma queda separado físicamente del código de la clase del programa.

Implementad ahora los métodos de la clase Complex mostrados en la Figura 1. **CADA VEZ QUE COMPLETÉIS UN MÉTODO DE DICHA CLASE, AÑADID EN LA CLASE Test CÓDIGO PARA INVOCARLO (UNA O VARIAS VECES DEPENDIENDO DE LOS CASOS QUE ENTENDÁIS DEBÉIS PROBAR) Y COMPROBAR QUE SU COMPORTAMIENTO ES CORRECTO.** Podéis utilizar el depurador.

NO PROGRAMÉIS DE UNA SOLA TIRADA TODOS LOS MÉTODOS Y COMENCÉIS A PROBARLOS DESPUÉS.

Tarea 2

Añadiremos ahora la clase con el método main() al programa. Cread la clase Main en el package "edu.upc.etsetb.poo.math".

El método main debe solicitar al usuario la introducción de un comando. Dicho comando ordenará al programa la realización de una cierta operación aritmética con números complejos. El programa realizará la operación, mostrará el resultado y solicitará un nuevo comando.

La ejecución del programa acabará cuando el usuario introduzca "F" o "f".

A continuación se muestran los posibles comandos que el programa debe reconocer:

+ (3,4) (4,-5.5)

- (3,4) (4,-5)

* (3,4) (4,-5)

/ (3,4) (4,-5)

En cada uno de ellos se muestran las partes reales y las partes imaginarias de números complejos con dos números encerrados entre paréntesis y separados por comas. Así, (4,-5) representa al número 4-5j.

El primer comando ordena al programa crear dos objetos Complex para gestionar los números complejos encerrados entre paréntesis, sumarlos y mostrar el resultado por pantalla.

El segundo ordena realizar las mismas operaciones pero restando los números complejos. El tercer y cuarto comando ordenan realizar la multiplicación y la división de los números complejos.

ES MUY IMPORTANTE QUE LOS ÚNICOS ESPACIOS EN BLANCO QUE HAYA EN LAS LÍNEAS SEAN LOS QUE SEPARAN LOS PARÉNTESIS ENTRE SÍ Y LOS PARÉNTESIS DE LOS SIMBOLOS DE LAS OPERACIONES.

NOTA 1: para leer lo introducido del teclado utilizad la clase Scanner. Utilizad el método split() de String para fraccionar el comando en sus componentes y para obtener las representaciones textuales de los valores de la parte real y la parte imaginaria de cada número complejo. Utilizad el método substring() de String para eliminar los paréntesis.

NOTA 2: utilizad el método estático parseDouble(...) de la clase Double para generar un dato de tipo double a partir de un String. Por ejemplo Double.parseDouble("5.5") devuelve el valor de tipo double 5.5.

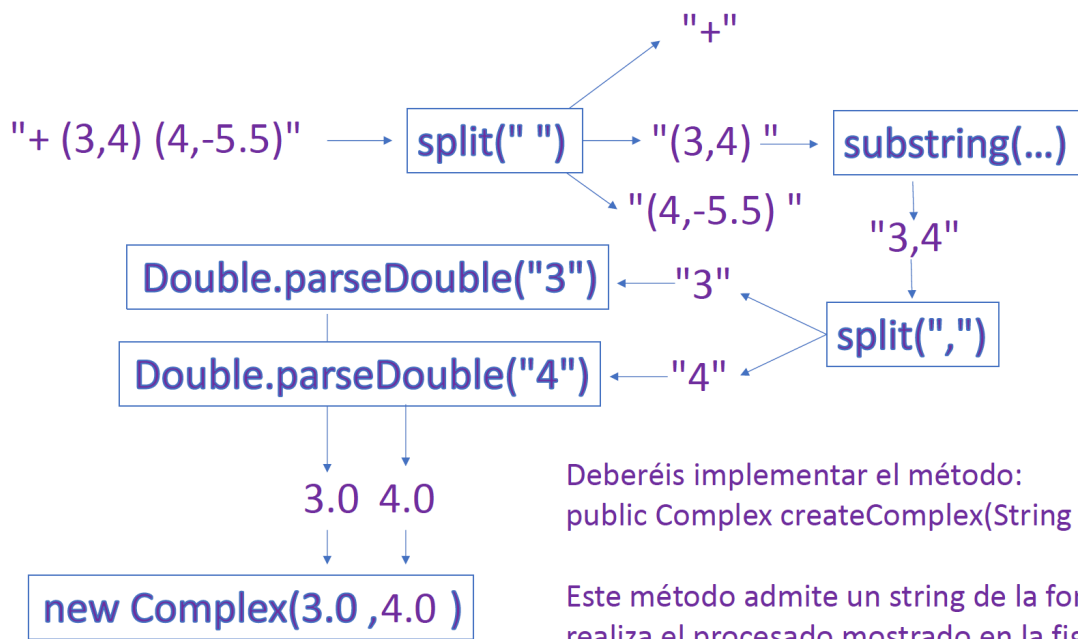
NOTA 3: **NO escribáis todo el código en el método main().** En su lugar cread en la clase Main un método para cada uno de los comandos, salvo para el comando de finalización. Estos métodos devolverán void y se limitarán a ejecutar las acciones antes reseñadas para cada comando. **Tened en cuenta que para poder invocar estos métodos deberéis crear un objeto de la clase Main.**

NOTA 4: Generad primero el código que finaliza la ejecución del programa, esto es, el tratamiento de "F" or "f". Para ello tened en cuenta que podéis utilizar los métodos toUpperCase() o toLowerCase() de String. Una vez el programa responde correctamente a la introducción de "F" o "f", tratad de resolver el caso de la suma. Cuando lo tengáis resuelto, el código para tratar el resto de casos es muy parecido.

En la siguiente página encontraréis una figura que muestra la secuencia de operaciones que deberéis implementar para crear los objetos de clase Complex, invocando a los métodos String::split(...), String::substring(...) y Double::parseDouble(...). Si tenéis dudas acerca de cómo invocarlos, consultad el API de Java. Asimismo, debéis generar el método

```
public Complex createComplex(String strComplex)
```

que admita como argumento un String de la forma "(3,4)" y cree y devuelva el objeto Complex correspondiente a la representación textual que define el String pasado como argumento (en el caso anterior, 3+4j).



Deberéis implementar el método:
`public Complex createComplex(String strComplex) ;`

Este método admite un string de la forma `"(3,4)"` y realiza el procesamiento mostrado en la figura anterior a partir de la invocación del método `substring(...)`